SMART UV EXPOSURE MONITORING SYSTEM

Prepared by
Fan Kai Che

INTRODUCTION

In Malaysia tropical climate, daily exposure to intense ultraviolet (UV) radiation poses significant health threats, including skin damage, sunburn, and elevated risks of skin cancer. UV is a part of the electromagnetic (light) spectrum that reaches the Earth from the sun with wavelengths shorter than visible spectrum. [2] The National Cancer Registry reported a rising incidence rate of skin cancer, underscoring the critical need for better awareness and preventive measures. [1] A critical gap exits in public awareness, as most people lack real-time tools to gauge personal UV exposure, relying instead on vague weather apps or intuition, which often prove inadequate against Malaysia's consistently high UV index.

The "Smart UV Exposure Monitoring System" is provide data on the current UV index, safe time limits based on skin type, and predictive warnings using location and weather inputs. The UV index describes the level of UV radiation at the Earth surface. The values of the index range from zero upward, which means the higher the index value, the greater the potential for damage to the skin and eye, and the less time it takes for harm to occurs. This innovation empowers individuals to make informed decisions, reducing health risks while promoting proactive sun safety in a high UV environment.

PROBLEM STATEMENT

1. Many people are not aware of the daily UV radiation levels and the health risks associated with prolonged exposure.
2. Individuals often engage in outdoor activities without considering UV intensity and index, leading to unsafe exposure during peak radiation hours.
3. Lack of accessible systems that provide real-time UV index data and personalized safety recommendations.

OBJECTIVE

1. To design an application that tracks and visualizes UV index levels in real-time for selected locations in Malaysia.
2. To apply physics-based models to estimate safe sun exposure time based on UV intensity.
3. To generate personalized alerts and protective guidelines, helping users reduce harmful UV exposure during high-risk periods.

RESULTS

Section 1 - Current UV Index

Section 2 - UV Index Forecast

Section 3 - Protection

Section 4 - Best Outdoor Time

Section 5 - Alter Bar

IMPORT

1. Tkinter = Build GUI.
2. Matplotlib.pyplot = Create the correct index gauge and forecast charts.
3. Matplotlib.patches = Mainly for the colored wedge bands on the gauge.
4. FigureCanvasTkAgg = Embeddedds matplotlib charts inside the tkinter window.
5. LineCollection = Draw the multi-colour forecast line.
6. Numpy = Generate hour arrays, interpolates current UVI.
7. Datatime = Gets current time and date.
8. Random = Adds small cloud variability noise to simulated UVI.

There are are fixed physical values used throughout the calculations:

1. The total solar power reaching the top of Earth's atmosphere is 1361.0 W/m^2.
2. There are only 0.07 (7 %) of sunlight is ultraviolet radiation.
3. The normalized ozone layer depth is 0.30.
4. The total value of UV is lost to Rayleigh scattering (air molecules) and dust is assume as 0.12.
5. Calibration factor for UVI value is 5.69.

Function 1: def solar_zenith_angle ( )

```
def solar_zenith_angle(hour, latitude_deg=3.1, day_of_year=None):
    if day_of_year is None:
        day_of_year = datetime.now().timetuple().tm_yday
# It reads today's day number from the system clock.

delta = 23.45 * math.sin(math.radians((360 / 365) * (day_of_year - 81)))
# It change throughout the year because Earth is tilted 23.45° . Day 81 is the March equinox (zero point). The copper equation is used to calculate.

H = (hour - 12.0) * 15.0
# To calculate how far the sun has removed from solar noon. At noon H = 0°, each hour  is approximate to 15 °.
```

Function 2 : def beer_lambert_uv ( )

```
def beer_lambert_uv(zenith_deg):
    if zenith_deg >= 90.0:
        return 0.0
# If the sun is at or below the horizon (θ ≥ 90°), there is zero UV. It will return 0 immediately.

cos_theta = math.cos(math.radians(zenith_deg))
    m = 1.0 / (cos_theta + 0.50572 * (96.07995 - zenith_deg) ** -1.6364)
# This is how much atmosphere the sunlight must travel through. The Kasten-Young Formula is used to calculate the optical mass mass.

k = ATMOS_EXTINCTION + OZONE_THICKNESS
    I_uv = SOLAR_CONSTANT * UV_FRACTION * cos_theta * math.exp(-k * m)
    return max(0.0, I_uv)
# This is Beer-Lambert Law
#  exp(-k * m) = Exponential decay. The more atmosphere light passes through, the more UV is absorbed.
```

Function 3 : def uv_irradiance_to-index ( )

Convert raw W/m^2 into UV index (UVI) fro 0 until 11+. It will divide by 5.69 so that the maximum tropical noon maps to UVI approximate 11.

Function 4 : def safe_exposure_minutes ( )

```
def safe_exposure_minutes(uvi, skin_type=3):
    med_table = {1: 100, 2: 200, 3: 300, 4: 450, 5: 600, 6: 800}
    MED = med_table.get(skin_type, 300)
```

MED stands for Minimal Erythermal Dose. The minimum UV dose that causes skin to redden (burn). Varies by Fitzpatrick skin type fro type 1 (very fair) burns fastest at 100 J/m^2, type 6 (dark) is most resistant at 800 J/m^2.

```
E_eff    = uvi * 25.0   # converts UVI to effective UV irradiance
    safe_sec = MED / (E_eff * 1e-3)   #Gives seconds until skin damage
    return safe_sec / 60.0
```

The safe time is calculated by using MED model.

Function 5 : def simulate_daily_uvi ( )

```
def simulate_daily_uvi(latitude_deg=3.1, cloud_factor=0.85, resolution_minutes=15):
    doy   = datetime.now().timetuple().tm_yday
    hours = np.arange(0, 24, resolution_minutes / 60.0)
```

Gets today's day-of-year and creates a time array from 0 to 24 in 15 minutes steps.

Function 6 : def uvi_category ( )

The dashboard will be categorize the UVI range and used everywhere in the dashboard to colour-code elements consistently.

<u>Class : UVDashboard</u>

def_ init _ ( )

Set up the window, including the title bar text, background, starting size pixel.

Stores simulation parameters.

#Generates the full day of UVI data. The np.interp is used to fin the exact UVI at the current minute by interpolating between the 15 minute data points.

def_ buld_ui ( )

Calls out the three layout sections (header, body, alert bar)

def_build_header ( )

Creates a dark strip 64 pixels tall across the full top of the window.

Creates a right-aligned sub-frame for location name and real time date and time.

def_build_body ( )

Creates the main content area using grid layout with 4 columns.

The weights [2,3,2,2] mean the forecast column have 3 shares of width. The "expand = True" is give the body fill all available space.

def_card ( )

The outer frames uses BORDER colour as visible thin border. The title label sits in that border area.

The inner frame uses BG_CARD is the content area.

def_uv_gauge ( )

Creates the first column, then creates a matplotlib figure with matching dark background. The "axis('off') is hides axes since this is a custom gauge, not a standard chart.

Creates five coloured (Low for green, Moderate for yellow, High for orange, very high for red and Extreme for purple)

Each band is a Wedge patch.

def_buid_forecast ( )

Plots UV index vs time graph

def_protection ( )

Risk banner label colored by current UVI category, followed by a thin 2px coloured horizontal line as a divider.

def_build_outdoor-time ( )

Scans through all the UVI data between 5 AM to 8 PM.

When UVI drops below 5, it will records the window start.

When it rises back to 5+, the will close the window and save the (start, end) pair.

The "if in_win" at the end catches a window still open at 8PM.

def_build_alert_bar ( )

Show the alert bar at the bottom of window.

Danger (red), Caution (orange), safe (green).

def_update_date ( )

Updates the header date/ time label with the current time.

def_schedle-refresh ( )

Refresh loop for every 30 seconds

REFERENCES

[1] https://www.sunwaymedicalvelocity.com.my/id/blogs/malaysians-advised-to-use-sunscreen-to-protect-against-harmful-uv-rays/

[2] https://www.met.gov.my/en/pendidikan/indeks-ultra-ungu/